

Agent Desk — Architecture

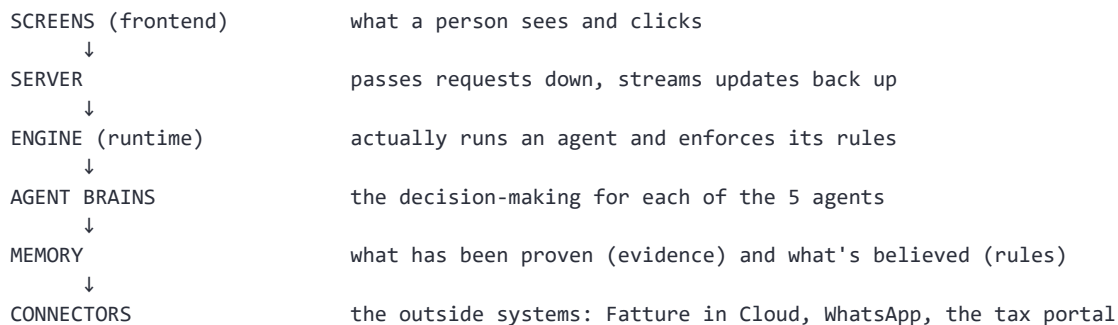
What this project is

A small AI system that helps an accounting firm (a "commercialista") check a client's VAT return before it's filed. It reads the numbers, fixes what it can, asks the client for anything missing (over WhatsApp), remembers the fix for next time — and then stops and waits for a human to press send. The AI is never allowed to file taxes by itself. That one rule is the most important thing in the whole system.

This document describes the pieces of the code and what each one is for — not who is building which part.

The big picture

Think of it as a stack, top to bottom. A request comes in at the top and flows down; anything the system learns flows back up into memory.



Two files sit outside this stack and above everything else: the contracts. Every other file has to agree with them — they're the shared rulebook.

contracts/ — the shared rulebook

File	What it's for
<code>manifestSchema.mjs</code>	Defines what a valid "agent description" must contain: what it's allowed to do, what it must refuse, when it must stop and ask a human, how it's paid. If a description is missing something required, this rejects it.
<code>a2aSchema.mjs</code>	Defines the 8 message types agents are allowed to send each other (e.g. "a document was delivered," "something is missing," "here is my answer"). No free-form chatting between agents — only these 8 shapes, each one signed so it can be trusted.
<code>seats/*.job.txt + *.json</code>	One pair per agent: the plain-language job description (<code>.job.txt</code>) and the compiled, structured version of it (<code>.json</code>) that the engine actually runs.

backend/runtime/ — the engine that runs an agent

File	What it's for
<code>openclaw.mjs</code>	Takes one compiled agent and actually runs it — loads its rules, gives it its allowed tools, lets it act. Without this, an agent description is just a document; this is what brings it to life.
<code>guardrails.mjs</code>	Double-checks every action an agent tries to take against its "must never do this" list and its "must ask a human first" rule — enforced here in code, not left to the AI to remember on its own.
<code>bus.mjs</code>	The mailroom: carries the 8 typed messages between agents, reliably, so a message isn't lost if something restarts.
<code>modelGateway.mjs</code>	The one place that knows how to ask an AI model a question and get a structured answer back. A small, local model handles most requests; a bigger model steps in as backup. Every other file that needs "AI thinking" calls through here, instead of each having its own way of talking to a model.

backend/ — the agent brains

File	Agent	What it's for
<code>validator.mjs</code>	L'Addetto IVA (VAT clerk)	Checks a VAT batch: is anything missing, does anything look wrong compared to last time. Sorts problems into a "needs a category" pile and a "missing document" pile.
<code>classifier.mjs</code>	Il Classificatore (Classifier)	Takes a line with a "needs a category" problem and figures out which accounting category it belongs to, with a confidence score. Below a certain confidence, it asks a human instead of guessing.
<code>amministrativo.mjs</code>	L'Amministrativo (Client Assistant)	Runs on the client's side. Decides what paperwork is missing for the period, and whether to fetch it automatically or ask the business owner directly on WhatsApp.
<code>smistatore.mjs</code>	Lo Smistatore (Dispatcher)	The traffic controller. Every message coming in — from a person or another agent — gets routed to the right place, once, based on who's responsible and available. Never answers the client itself.
<code>archivista.mjs</code>	L'Archivista (Archivist)	Whenever a human confirms a fix, turns it into a reusable rule ("this supplier always goes in this category") and saves it — so the same problem doesn't need to be solved twice. New rules run quietly for a while before being trusted.

File	Agent	What it's for
compiler.mjs	Chief of Staff	Reads an agent's plain-language job description and turns it into the structured, checked version the engine can run. This is the "translator" between a human writing instructions and the system understanding them.

backend/connectors/ — talking to the outside world

File	What it's for
fattureInCloud.mjs	Reads invoices and client records from Fatture in Cloud, and writes the classified category back once decided.
teamSystem.mjs	Reads the VAT batch from the accounting ledger, and writes the filing status back once the human has signed and sent it.
whatsapp.mjs	Sends and receives WhatsApp messages between L'Amministrativo and the business owner.
adePortal.mjs	Prepares a filing for the tax authority — but its "send" action is permanently disabled by design. A human always does the actual sending.
bankFeed.mock.mjs	A stand-in for a real bank connection, used for now so the demo doesn't need a live bank account.

backend/memory/ — what the system remembers

File	What it's for
evidenceStore.mjs	An unchangeable log of facts and proof (e.g. "this exact invoice arrived on this date"). Nothing is ever edited or deleted here, only added.
knowledgeStore.mjs	The rules the system currently believes are true (e.g. "this supplier → this category"), each one tagged with how confident we are and who confirmed it. Rules can be updated as confidence changes.

backend/scenario/vatFilingPath.mjs — the demo script

Plays the whole story from start to finish, in order: batch arrives → checked → low-confidence lines resolved → missing item chased through the client → rule saved → batch re-assembled → human gate → filed. Useful both as a live demo and as a plain description of how the pieces are meant to connect.

frontend/ — the 5 screens

Screen	What it shows
Roster	The list of agents, like a staff directory.
Agent page	One agent's job description, editable in plain language, with a button to compile it.
A2A stream	The typed messages agents send each other, as a chat log.
Board	The whole process playing out, step by step, live.
Memory panel	What's been proven (evidence) and what's been learned (rules), live.

tests/

One test per module above, checking two things at minimum: does its output still match the contracts, and does the full demo story still complete from start to finish without breaking. The demo path is the simplest possible safety net — if it stops completing, something upstream broke.